

Database Systems Laboratory Work — Week 2

Part 1: Key Identification Exercises

Task 1.1 — Relation A: Employee

Relation:

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

From the sample data, **EmpID, SSN, Email, and Phone are each unique**.

Therefore, assuming the business rules guarantee their uniqueness, each can identify an employee.

1. At least 6 superkeys

A **superkey** is any set of attributes that uniquely identifies a tuple.

Examples:

1. {EmpID}
2. {SSN}
3. {Email}
4. {Phone}
5. {EmpID, Name}
6. {EmpID, Department}
7. {SSN, Name}
8. {Email, Phone}
9. {EmpID, SSN}
10. {EmpID, SSN, Email}

There are many more. In fact, **any attribute set containing a candidate key is a superkey**.

2. Candidate keys

The candidate keys are the minimal unique identifiers:

- {EmpID}
- {SSN}
- {Email}
- {Phone}

They are candidate keys because each uniquely identifies an employee and contains no unnecessary attribute.

Important: The sample data alone does not prove that Phone will *always* be unique. This conclusion requires a business rule saying phone numbers are unique. If no such rule exists, Phone should not be considered a candidate key.

3. Choice of primary key

I would choose **EmpID** as the primary key.

Reasons:

- It is specifically designed to identify employees.
- It is short and simple.
- It does not contain personal information.
- SSN is sensitive personal information and should generally not be used as the primary identifier.
- Email and phone numbers can change.
- EmpID can remain stable throughout employment.

Therefore:

Employee(**EmpID**, SSN, Email, Phone, Name, Department, Salary)

4. Can two employees have the same phone number?

Based only on the displayed data, no two employees currently have the same phone number.

However, the data does **not** establish that phone must be unique in the future.

For example, two employees might share:

Phone = 555-0101

unless the database has a UNIQUE constraint or a business rule stating that every employee must have a unique phone number.

Relation B — Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business rules:

- A student may take the same course in different semesters.
- A student cannot register for the same course section in the same semester.
- A section has a fixed credit value.

1. Minimum attributes for the primary key

The minimum key is:

(StudentID, CourseCode, Section, Semester, Year)

This is a composite primary key.

2. Why each attribute is necessary

- **StudentID** — identifies which student is registered.
- **CourseCode** — identifies the course.
- **Section** — distinguishes different sections of the same course.
- **Semester** — allows the student to take the same course again in another semester.
- **Year** — distinguishes the same semester in different academic years.

For example:

(101, CS101, 1, Fall, 2025)

and

(101, CS101, 1, Fall, 2026)

must represent different registrations.

3. Additional candidate keys

No additional candidate key can be inferred from the stated business rules.

Credits cannot be a key because many registrations can have the same number of credits.

Grade cannot be a key because many students can receive the same grade.

Task 1.2 — Foreign Key Design

Given:

- Student(StudentID, Name, Email, Major, AdvisorID)
- Professor(ProfID, Name, Department, Salary)
- Course(CourseID, Title, Credits, DepartmentCode)
- Department(DeptCode, DeptName, Budget, ChairID)
- Enrollment(StudentID, CourseID, Semester, Grade)

Foreign keys

1. Student → Professor

Student.AdvisorID → Professor.ProfID

This represents the student's advisor.

2. Course → Department

Course.DepartmentCode → Department.DeptCode

This identifies the department offering the course.

3. Department → Professor

Department.ChairID → Professor.ProfID

This identifies the professor serving as department chair.

4. Enrollment → Student

Enrollment.StudentID → Student.StudentID

5. Enrollment → Course

Enrollment.CourseID → Course.CourseID

Therefore:

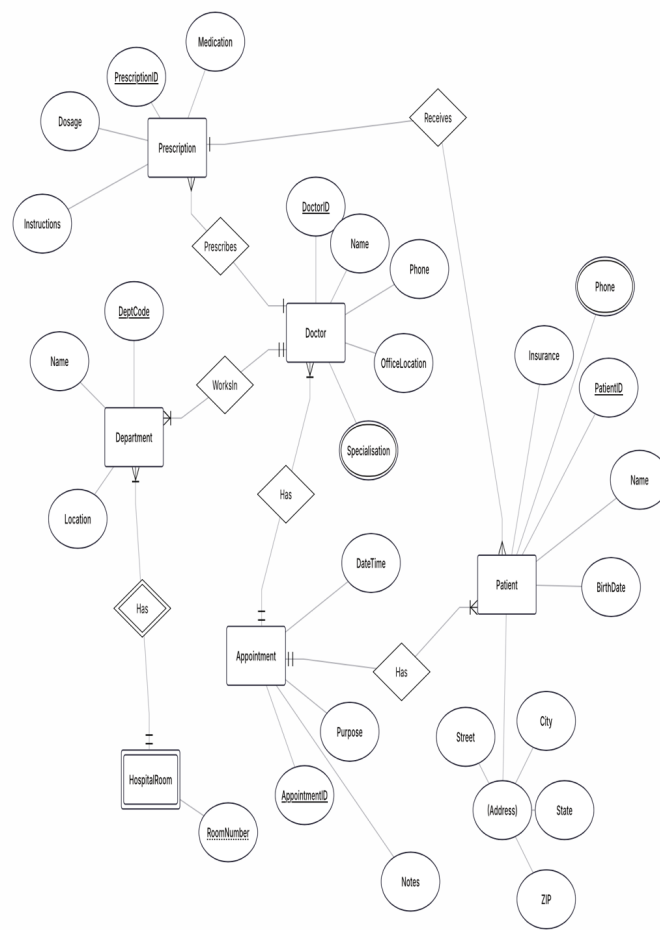
Student AdvisorID —————→ Professor (ProfID)

Course DepartmentCode —————→ Department (DeptCode)

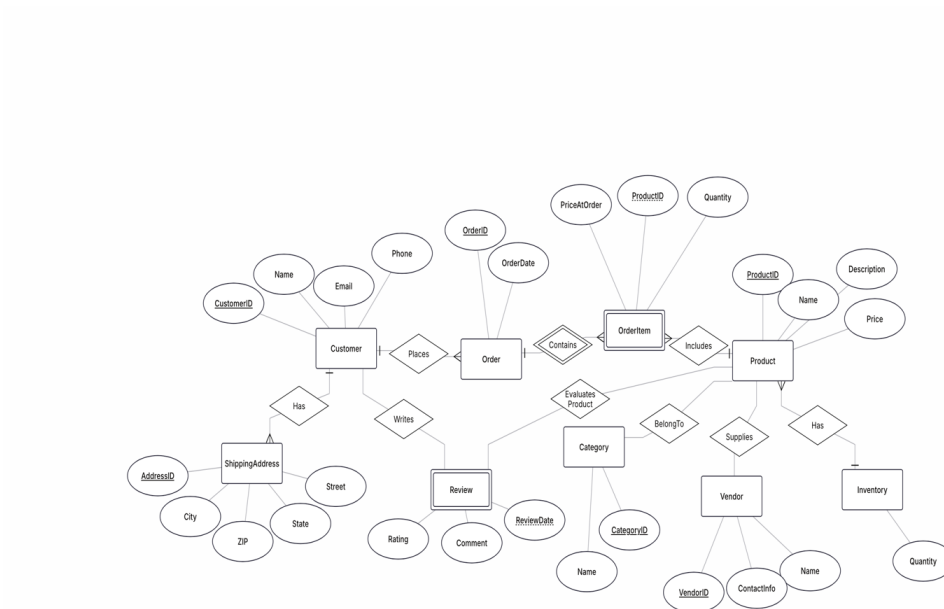
Department ChairID —————→ Professor (ProfID)

Enrollment StudentID —————→ Student(StudentID)

Task 2.1 Hospital Management System



Task 2.2: E - commerce Platform



Part 4: Normalization Workshop Task

4.1: Denormalized Table Analysis

1. Functional Dependencies

StudentID → StudentName, StudentMajor

ProjectID → ProjectTitle, ProjectType, SupervisorID

SupervisorID → SupervisorName, SupervisorDept

(StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2. Problems

Redundancy:

Student information is repeated for every project.

Project information is repeated for every student working on it.

Supervisor information is repeated for every related project record.

Update anomaly:

If a supervisor changes their department, all records containing that supervisor must be updated.

Insert anomaly:

A new project cannot be inserted if no student is assigned to it.

Delete anomaly:

Deleting the last student assigned to a project may also delete the only record containing information about that project and supervisor.

3. First Normal Form (1NF)

The table is already in 1NF because all attributes contain atomic values and there are no repeating groups.

4. Second Normal Form (2NF)

Primary Key:

(StudentID, ProjectID)

Partial Dependencies:

- StudentID $\rightarrow$ StudentName, StudentMajor
- ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID

2NF Decomposition:

Student(StudentID PK, StudentName, StudentMajor)

Project(ProjectID PK, ProjectTitle, ProjectType, SupervisorID)

StudentProject(StudentID PK/FK, ProjectID PK/FK, Role, HoursWorked, StartDate, EndDate)

5. Third Normal Form (3NF)

Transitive Dependency:

ProjectID $\rightarrow$ SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Therefore, supervisor information must be separated.

Final 3NF Decomposition:

Student(StudentID PK, StudentName, StudentMajor)

Project(ProjectID PK, ProjectTitle, ProjectType, SupervisorID FK)

Supervisor(SupervisorID PK, SupervisorName, SupervisorDept)

StudentProject(StudentID PK/FK, ProjectID PK/FK, Role, HoursWorked, StartDate, EndDate)

Task 4.2: Advanced Normalization

1. Primary Key

Primary Key: (StudentID, CourseID)

A student can enroll in multiple courses, and a course can have multiple students.

2. Functional Dependencies

- StudentID $\rightarrow$ StudentMajor
- CourseID $\rightarrow$ CourseName
- InstructorID $\rightarrow$ InstructorName
- CourseID $\rightarrow$ InstructorID, TimeSlot, Room
- (Room, TimeSlot) $\rightarrow$ Building

3. BCNF Check

The table is not in BCNF.

For example:

StudentID $\rightarrow$ StudentMajor

StudentID is not a superkey because one student can have multiple courses.

Also:

CourseID $\rightarrow$ CourseName

CourseID is not a superkey because one course can have multiple students.

Therefore, the BCNF condition is violated.

4. BCNF Decomposition

Student(StudentID PK, StudentMajor)

Course(CourseID PK, CourseName)

Instructor(InstructorID PK, InstructorName)

CourseSection(CourseID PK, InstructorID FK, TimeSlot, Room)

RoomSchedule(Room PK, TimeSlot PK, Building)

Enrollment(StudentID PK/FK, CourseID PK/FK)

5. Information Loss

The decomposition is lossless if the stated functional dependencies are correct.

The original information can be reconstructed by joining the decomposed tables using their primary and foreign keys.

If the dependencies or keys are defined incorrectly, joining the tables could produce duplicate or incorrect combinations of data.

Task 5.1. Real-World Application

